

820

C++语言的设计者希望它能处理各种各样的问题。因此，C++的某些特征可能对于一些特殊的应用非常重要，而在另外一些情况下没什么作用。本章将介绍 C++语言的几种未被广泛使用的特征。

19.1 控制内存分配

某些应用程序对内存分配有特殊的需求，因此我们无法将标准内存管理机制直接应用于这些程序。它们常常需要自定义内存分配的细节，比如使用关键字 `new` 将对象放置在特定的内存空间中。为了实现这一目的，应用程序需要重载 `new` 运算符和 `delete` 运算符以控制内存分配的过程。

19.1.1 重载 `new` 和 `delete`

尽管我们说能够“重载 `new` 和 `delete`”，但是实际上重载这两个运算符与重载其他运算符的过程大不相同。要想真正掌握重载 `new` 和 `delete` 的方法，首先要对 `new` 表达式和 `delete` 表达式的工作机理有更多了解。

当我们使用一条 `new` 表达式时：

```
// new 表达式
string *sp = new string("a value");    // 分配并初始化一个 string 对象
string *arr = new string[10];           // 分配 10 个默认初始化的 string 对象
```

实际执行了三步操作。第一步，`new` 表达式调用一个名为 `operator new`（或者 `operator new[]`）的标准库函数。该函数分配一块足够大的、原始的、未命名的内存空间以便存储特定类型的对象（或者对象的数组）。第二步，编译器运行相应的构造函数以构造这些对象，并为其传入初始值。第三步，对象被分配了空间并构造完成，返回一个指向该对象的指针。

当我们使用一条 `delete` 表达式删除一个动态分配的对象时：

```
delete sp;                                // 销毁*sp，然后释放 sp 指向的内存空间
delete [] arr;                            // 销毁数组中的元素，然后释放对应的内存空间
```

实际执行了两步操作。第一步，对 `sp` 所指的或者 `arr` 所指的数组中的元素执行对应的析构函数。第二步，编译器调用名为 `operator delete`（或者 `operator delete[]`）的标准库函数释放内存空间。

如果应用程序希望控制内存分配的过程，则它们需要定义自己的 `operator new` 函数和 `operator delete` 函数。即使在标准库中已经存在这两个函数的定义，我们仍旧可以定义自己的版本。编译器不会对这种重复的定义提出异议，相反，编译器将使用我们自定义的版本替换标准库定义的版本。

821



当自定义了全局的 `operator new` 函数和 `operator delete` 函数后，我们就担负起了控制动态内存分配的职责。这两个函数必须是正确的：因为它们是程序整个处理过程中至关重要的一部分。

应用程序可以在全局作用域中定义 `operator new` 函数和 `operator delete` 函数，也可以将它们定义为成员函数。当编译器发现一条 `new` 表达式或 `delete` 表达式后，将

在程序中查找可供调用的 `operator` 函数。如果被分配（释放）的对象是类类型，则编译器首先在类及其基类的作用域中查找。此时如果该类含有 `operator new` 成员或 `operator delete` 成员，则相应的表达式将调用这些成员。否则，编译器在全局作用域查找匹配的函数。此时如果编译器找到了用户自定义的版本，则使用该版本执行 `new` 表达式或 `delete` 表达式；如果没找到，则使用标准库定义的版本。

我们可以使用作用域运算符 `new` 表达式或 `delete` 表达式忽略定义在类中的函数，直接执行全局作用域中的版本。例如，`::new` 只在全局作用域中查找匹配的 `operator new` 函数，`::delete` 与之类似。

`operator new` 接口和 `operator delete` 接口

标准库定义了 `operator new` 函数和 `operator delete` 函数的 8 个重载版本。其中前 4 个版本可能抛出 `bad_alloc` 异常，后 4 个版本则不会抛出异常：

```
// 这些版本可能抛出异常
void *operator new(size_t);           // 分配一个对象
void *operator new[](size_t);        // 分配一个数组
void *operator delete(void*) noexcept; // 释放一个对象
void *operator delete[](void*) noexcept; // 释放一个数组

// 这些版本承诺不会抛出异常，参见 12.1.2 节（第 409 页）
void *operator new(size_t, nothrow_t&) noexcept;
void *operator new[](size_t, nothrow_t&) noexcept;
void *operator delete(void*, nothrow_t&) noexcept;
void *operator delete[](void*, nothrow_t&) noexcept;
```

类型 `nothrow_t` 是定义在 `new` 头文件中的一个 `struct`，在这个类型中不包含任何成员。`new` 头文件还定义了一个名为 `nothrow` 的 `const` 对象，用户可以通过这个对象请求 `new` 的非抛出版本（参见 12.1.2 节，第 408 页）。与析构函数类似，`operator delete` 也不允许抛出异常（参见 18.1.1 节，第 685 页）。当我们重载这些运算符时，必须使用 `noexcept` 异常说明符（参见 18.1.4 节，第 690 页）指定其不抛出异常。

应用程序可以自定义上面函数版本中的任意一个，前提是自定义的版本必须位于全局作用域或者类作用域中。当我们上述运算符函数定义成类的成员时，它们是隐式静态的（参见 7.6 节，第 270 页）。我们无须显式地声明 `static`，当然这么做也不会引发错误。因为 `operator new` 用在对象构造之前而 `operator delete` 用在对象销毁之后，所以这两个成员（`new` 和 `delete`）必须是静态的，而且它们不能操纵类的任何数据成员。

822

对于 `operator new` 函数或者 `operator new[]` 函数来说，它的返回类型必须是 `void*`，第一个形参的类型必须是 `size_t` 且该形参不能含有默认实参。当我们为一个对象分配空间时使用 `operator new`；为一个数组分配空间时使用 `operator new[]`。当编译器调用 `operator new` 时，把存储指定类型对象所需的字节数传给 `size_t` 形参；当调用 `operator new[]` 时，传入函数的则是存储数组中所有元素所需的空間。

如果我们想要自定义 `operator new` 函数，则可以为它提供额外的形参。此时，用到这些自定义函数的 `new` 表达式必须使用 `new` 的定位形式（参见 12.1.2 节，第 409 页）将实参传给新增的形参。尽管在一般情况下我们可以自定义具有任何形参的 `operator new`，但是下面这个函数却无论如何不能被用户重载：

```
void *operator new(size_t, void*);           // 不允许重新定义这个版本
```

这种形式只供标准库使用，不能被用户重新定义。

对于 `operator delete` 函数或者 `operator delete[]` 函数来说，它们的返回类型必须是 `void`，第一个形参的类型必须是 `void*`。执行一条 `delete` 表达式将调用相应的 `operator` 函数，并用指向待释放内存的指针来初始化 `void*` 形参。

当我们将 `operator delete` 或 `operator delete[]` 定义成类的成员时，该函数可以包含另外一个类型为 `size_t` 的形参。此时，该形参的初始值是第一个形参所指对象的字节数。`size_t` 形参可用于删除继承体系中的对象。如果基类有一个虚析构函数（参见 15.7.1 节，第 552 页），则传递给 `operator delete` 的字节数将因待删除指针所指对象的动态类型不同而有所区别。而且，实际运行的 `operator delete` 函数版本也由对象的动态类型决定。

术语：new 表达式与 operator new 函数

标准库函数 `operator new` 和 `operator delete` 的名字容易让人误解。和其他 `operator` 函数不同（比如 `operator=`），这两个函数并没有重载 `new` 表达式或 `delete` 表达式。实际上，我们根本无法自定义 `new` 表达式或 `delete` 表达式的行为。

一条 `new` 表达式的执行过程总是先调用 `operator new` 函数以获取内存空间，然后在得到的内存空间中构造对象。与之相反，一条 `delete` 表达式的执行过程总是先销毁对象，然后调用 `operator delete` 函数释放对象所占的空间。

我们提供新的 `operator new` 函数和 `operator delete` 函数的目的在于改变内存分配的方式，但是不管怎样，我们都不能改变 `new` 运算符和 `delete` 运算符的基本含义。

823 > malloc 函数与 free 函数

当你定义了自己的全局 `operator new` 和 `operator delete` 后，这两个函数必须以某种方式执行分配内存与释放内存的操作。也许你的初衷仅仅是使用一个特殊定制的内存分配器，但是这两个函数还应该同时满足某些测试的目的，即检验其分配内存的方式是否与常规方式类似。

为此，我们可以使用名为 **malloc** 和 **free** 的函数，C++ 从 C 语言中继承了这些函数，并将其定义在 `cstdlib` 头文件中。

`malloc` 函数接受一个表示待分配字节数的 `size_t`，返回指向分配空间的指针或者返回 0 以表示分配失败。`free` 函数接受一个 `void*`，它是 `malloc` 返回的指针的副本，`free` 将相关内存返回给系统。调用 `free(0)` 没有任何意义。

如下所示是编写 `operator new` 和 `operator delete` 的一种简单方式，其他版本与之类似：

```
void *operator new(size_t size) {
    if (void *mem = malloc(size))
        return mem;
    else
        throw bad_alloc();
}

void operator delete(void *mem) noexcept { free(mem); }
```

19.1.1 节练习

练习 19.1: 使用 `malloc` 编写你自己的 `operator new(size_t)` 函数, 使用 `free` 编写 `operator delete(void *)` 函数。

练习 19.2: 默认情况下, `allocator` 类使用 `operator new` 获取存储空间, 然后使用 `operator delete` 释放它。利用上一题中的两个函数重新编译并运行你的 `StrVec` 程序 (参见 13.5 节, 第 465 页)。

19.1.2 定位 new 表达式

尽管 `operator new` 函数和 `operator delete` 函数一般用于 `new` 表达式, 然而它们毕竟是标准库的两个普通函数, 因此普通的代码也可以直接调用它们。

在 C++ 的早期版本中, `allocator` 类 (参见 12.2.2 节, 第 427 页) 还不是标准库的一部分。应用程序如果想把内存分配与初始化分离开来的话, 需要调用 `operator new` 和 `operator delete`。这两个函数的行为与 `allocator` 的 `allocate` 成员和 `deallocate` 成员非常类似, 它们负责分配或释放内存空间, 但是不会构造或销毁对象。

与 `allocator` 不同的是, 对于 `operator new` 分配的内存空间来说我们无法使用 `construct` 函数构造对象。相反, 我们应该使用 `new` 的 **定位 new** (`placement new`) 形式 (参见 12.1.2 节, 第 409 页) 构造对象。如我们所知, `new` 的这种形式为分配函数提供了额外的信息。我们可以使用定位 `new` 传递一个地址, 此时定位 `new` 的形式如下所示:

824

```
new (place_address) type
new (place_address) type (initializers)
new (place_address) type [size]
new (place_address) type [size] { braced initializer list }
```

其中 `place_address` 必须是一个指针, 同时在 `initializers` 中提供一个 (可能为空的) 以逗号分隔的初始值列表, 该初始值列表将用于构造新分配的对象。

当仅通过一个地址值调用时, 定位 `new` 使用 `operator new(size_t, void*)` “分配” 它的内存。这是一个我们无法自定义的 `operator new` 版本 (参见 19.1.1 节, 第 727 页)。该函数不分配任何内存, 它只是简单地返回指针实参; 然后由 `new` 表达式负责在指定的地址初始化对象以完成整个工作。事实上, 定位 `new` 允许我们在一个特定的、预先分配的内存地址上构造对象。



当只传入一个指针类型的实参时, 定位 `new` 表达式构造对象但是不分配内存。

尽管在很多时候使用定位 `new` 与 `allocator` 的 `construct` 成员非常相似, 但在它们之间也有一个重要的区别。我们传给 `construct` 的指针必须指向同一个 `allocator` 对象分配的空间, 但是传给定位 `new` 的指针无须指向 `operator new` 分配的内存。实际上如我们将在 19.6 节 (第 753 页) 介绍的, 传给定位 `new` 表达式的指针甚至不需要指向动态内存。

显式的析构函数调用

就像定位 `new` 与使用 `allocate` 类似一样, 对析构函数的显式调用也与使用 `destroy` 很类似。我们既可以通过对象调用析构函数, 也可以通过对象的指针或引用调

用析构函数，这与调用其他成员函数没什么区别：

```
string *sp = new string("a value"); // 分配并初始化一个 string 对象
sp->~string();
```

在这里我们直接调用了析构函数。箭头运算符解引用指针 `sp` 以获得 `sp` 所指的对象，然后我们调用析构函数，析构函数的形式是波浪线（~）加上类型的名字。

和调用 `destroy` 类似，调用析构函数可以清除给定的对象但是不会释放该对象所在的空间。如果需要的话，我们可以重新使用该空间。



调用析构函数会销毁对象，但是不会释放内存。

825 19.2 运行时类型识别

运行时类型识别（run-time type identification, RTTI）的功能由两个运算符实现：

- `typeid` 运算符，用于返回表达式的类型。
- `dynamic_cast` 运算符，用于将基类的指针或引用安全地转换成派生类的指针或引用。

当我们将这两个运算符用于某种类型的指针或引用，并且该类型含有虚函数时，运算符将使用指针或引用所绑定对象的动态类型（参见 15.2.3 节，第 534 页）。

这两个运算符特别适用于以下情况：我们想使用基类对象的指针或引用执行某个派生类操作并且该操作不是虚函数。一般来说，只要有可能我们应该尽量使用虚函数。当操作被定义成虚函数时，编译器将根据对象的动态类型自动地选择正确的函数版本。

然而，并非任何时候都能定义一个虚函数。假设我们无法使用虚函数，则可以使用一个 RTTI 运算符。另一方面，与虚成员函数相比，使用 RTTI 运算符蕴含着更多潜在的风险：程序员必须清楚地知道转换的目标类型并且必须检查类型转换是否被成功执行。



使用 RTTI 必须要加倍小心。在可能的情况下，最好定义虚函数而非直接接管类型管理的重任。

19.2.1 `dynamic_cast` 运算符

`dynamic_cast` 运算符（`dynamic_cast` operator）的使用形式如下所示：

```
dynamic_cast<type*>(e)
dynamic_cast<type&>(e)
dynamic_cast<type&&>(e)
```

其中，`type` 必须是一个类类型，并且通常情况下该类型应该含有虚函数。在第一种形式中，`e` 必须是一个有效的指针（参见 2.3.2 节，第 47 页）；在第二种形式中，`e` 必须是一个左值；在第三种形式中，`e` 不能是左值。

在上面的所有形式中，`e` 的类型必须符合以下三个条件中的任意一个：`e` 的类型是目标 `type` 的公有派生类、`e` 的类型是目标 `type` 的公有基类或者 `e` 的类型就是目标 `type` 的类型。如果符合，则类型转换可以成功。否则，转换失败。如果一条 `dynamic_cast` 语句的转换目标是指针类型并且失败了，则结果为 0。如果转换目标是引用类型并且失败了，